

RUNBOOK

APIC ProtoHub

How the system connects, and how to run it

For developers wiring the two front ends to the database via an API layer. Assumes a laptop with a terminal, and no prior setup.

1 • What you have right now

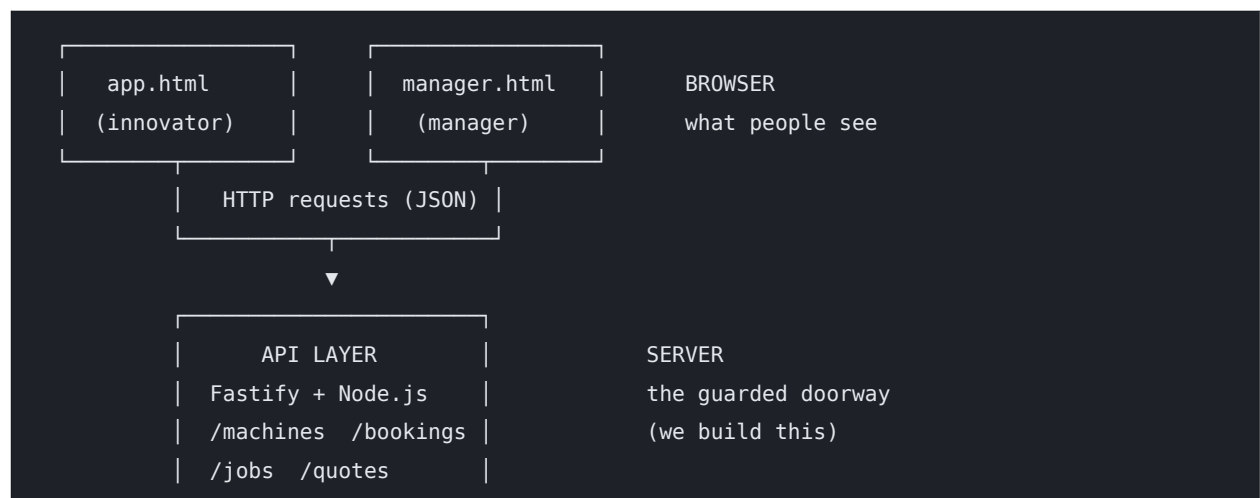
Four pieces exist today. Two are user interfaces, two are the database brain. They are NOT yet connected — the front ends currently run on their own copy of the data. This runbook is about wiring them together through a middle layer (an API) so they read and write the real database.

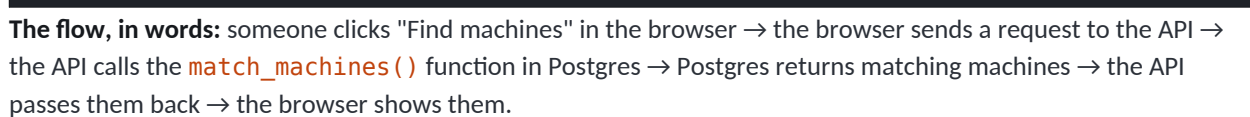
FILE	WHAT IT IS	ROLE
schema.sql	Tables: centres, machines, capabilities, bookings, jobs, quotes, event log	Database structure
match.sql	The capability matcher + free-slot finder	Search brain
manager.sql	Quote pricing, job state machine, stock ledger, dashboard views	Operations brain
seed.sql / seed_jobs.sql	Realistic test data — 5 centres, 12 machines, sample jobs	Demo data
app.html	Innovator front end — discover a machine, book a slot	User interface
manager.html	Manager console — queue, pricing, job tracking, stock	User interface

WHY AN API IN THE MIDDLE. A browser cannot talk to a database directly — that would expose the database to the whole internet. The API is a guarded doorway: the front ends ask it for things ("find machines", "price this job"), it runs the safe database function and returns the answer. This is the piece we are adding.

2 • How it fits together

Read this top to bottom — it is the whole system in one picture:





3 • Getting it running, step by step

Do these in order. Each step is a block you paste into a terminal. Lines starting with \$ are commands; lines starting with # are explanations.

1 Install PostgreSQL

The database engine. On Ubuntu/Debian:

```
# refresh package list, then install
$ sudo apt-get update
$ sudo apt-get install -y postgresql postgresql-contrib

# on Mac instead: brew install postgresql@16
```

2 Create the database and load the brain

Create an empty database, turn on the extension the booking constraint needs (**btree_gist**), then load the four SQL files in this exact order — each depends on the one before:

```
$ createdb protohub
$ psql protohub -c "CREATE EXTENSION btree_gist;"

# order matters: structure → search → operations → data
$ psql protohub -f schema.sql
$ psql protohub -f match.sql
$ psql protohub -f manager.sql
$ psql protohub -f seed.sql
$ psql protohub -f seed_jobs.sql
```

CHECK IT WORKED. Run `psql protohub -c "SELECT count(*) FROM machine;"` — you should see 12. If you see an error about `btree_gist`, the extension step was skipped.

3 Confirm the brain works — no API needed yet

You can call the database functions directly to prove they work before building anything on top. Try the matcher:

```
# "find a machine near Kakinada for a 120x80x40mm PLA part"
$ psql protohub -c "SELECT machine, centre_name, distance_km
  FROM match_machines(120,80,40,'pla',NULL,NULL,16.98,82.24,1);"

# price a job: issue_quote(job, manager, minutes, grams)
$ psql protohub -c "SELECT issue_quote(3, 3, 45, 40, 'PLA housing');"
```

If those return rows, the entire backend is proven. Everything else is just plumbing to reach these from a browser.

4 • Building the API layer (the missing piece)

This is the one part not yet built. It is a small Node.js program that listens for browser requests and translates them into database calls. Here is the minimum to stand it up.

1 Set up the project

```
$ mkdir protohub-api && cd protohub-api
$ npm init -y
$ npm install fastify @fastify/cors pg

# fastify = the web server    pg = talks to Postgres
# @fastify/cors = lets the browser call it
```

2 Write the server

Create a file `server.js`. This example exposes three endpoints — search, book, and the manager queue. The pattern repeats for every other function.

```
const Fastify = require("fastify");
const { Pool } = require("pg");

const db = new Pool({ connectionString:
  "postgres://localhost/protohub" });
const app = Fastify();
app.register(require("@fastify/cors"));

// SEARCH: browser sends part size + material, gets machines
app.get("/machines", async (req) => {
  const { x, y, z, mat, lat, lng } = req.query;
  const r = await db.query(
    "SELECT * FROM match_machines($1,$2,$3,$4,
      NULL,NULL,$5,$6,1)",
    [x, y, z, mat||null, lat, lng]);
  return r.rows;
});

// MANAGER QUEUE: what needs doing at a centre
app.get("/queue/:centre", async (req) => {
  const r = await db.query(
    "SELECT * FROM v_manager_queue WHERE centre_id=$1",
    [req.params.centre]);
  return r.rows;
});

// PRICE A JOB: manager sends time + grams
app.post("/jobs/:id/quote", async (req) => {
  const { minutes, grams } = req.body;
  const r = await db.query(
```

```

    "SELECT issue_quote($1,$2,$3,$4,$5) AS total",
    [req.params.id, 3, minutes, grams, "Quoted"]);
    return r.rows[0];
  });

app.listen({ port: 3000 });
console.log("API running on http://localhost:3000");

```

3 Run it

```

$ node server.js
# → API running on http://localhost:3000

# test it from another terminal:
$ curl "http://localhost:3000/machines?x=120&y=80&z=40&mat=pla&lat=16.98&lng=82.24"

```

THAT curl IS THE WHOLE POINT. When that command returns a list of machines as JSON, the browser can get the same thing. The API is now the working bridge between browser and database.

5 • Connecting the front ends

Today the two HTML files hold their data in a JavaScript variable near the top (look for `const MACHINES = [` in `app.html`, and `let JOBS = [` in `manager.html`). To go live, replace those hardcoded arrays with a call to the API.

BEFORE — hardcoded data

```

const MACHINES = [
  {id:1, make:"Ultimaker", model:"S5", ...},
  ... 12 machines typed out by hand
];
render(MACHINES);

```

AFTER — live from the API

```

async function search(x, y, z, mat, lat, lng) {
  const res = await fetch(
    `http://localhost:3000/machines?x=${x}&y=${y}` +
    `&z=${z}&mat=${mat}&lat=${lat}&lng=${lng}`);
  const machines = await res.json();
  render(machines); // same render function as before
}

```

The render functions do not change — they already expect exactly the shape the database returns, because both front ends were built to mirror the real views. That was deliberate: it makes this swap a few lines, not a rewrite.

DO THE MANAGER CONSOLE THE SAME WAY. Replace `manager.html`'s `JOBS` array with a fetch to `/queue/1`, and wire the "Send quote" and "Advance" buttons to POST to `/jobs/:id/quote` and `/jobs/:id/advance`. The pricing math already matches the database to the paisa, so numbers will not shift.

6 • When something breaks

SYMPTOM	LIKELY CAUSE	FIX
"type gist does not exist" when loading schema	btree_gist extension not enabled	Run the CREATE EXTENSION line from Step 2
Browser shows no machines, console says CORS error	@fastify/cors not registered	Confirm the app.register(cors) line is present
"connection refused" on port 3000	API server not running	Start it: node server.js
"illegal transition" when advancing a job	Working as designed — job not in a valid prior state	Check the job's current status first
Double-booking somehow got through	Cannot happen — the database rejects it	If seen, the constraint was dropped; reload schema.sql
psql: "database protohub does not exist"	Step 2 not run, or ran in wrong directory	Run createdb protohub

7 • Before real users touch it

The steps above give you a working system on a laptop. For an actual pilot, four things change — in rough order of importance:

- **Real data replaces seed data.** The single most important step. Get one APIC's actual machine list, rates, and hours. This will reveal small schema gaps no design can predict — expect to add a column or two.
- **Authentication.** Right now anyone can call any endpoint. Add phone-OTP login for innovators and a password/SSO for staff, and check the user's role before manager actions. The role field already exists in the app_user table.
- **Hosting.** Move Postgres and the API onto a server — AWS Mumbai (ap-south-1) keeps data in India. Put the two HTML files behind any static host. Nothing in the code assumes a laptop.
- **File uploads.** Innovators need to attach CAD files to jobs. Add an S3 bucket and store the file key in the existing job_file table. Until then, files are handled offline.